PI - Safepay RAAST - API Documentation/Testing

Document Number	OI - PI - **20260226**
Document status	DRAFT
Document owner	@Shayan Rasheed @Rizwan Zaffar
Solutions Architect	@Maqsood Ali @Owais Khalid @Saqlain Raza
Project Manager	TBD
Priority	TBD
Region	**Pakistan**
Operator	**RAAST-Safepay**
Product	PAYIN

Postman: **Raast Public.postman_collection.json**

Aggregator: agg_ca1e980e-a03f-41f4-83e8-aea94d6426ea
Aggregator Merchant: am_97a8f3d5-ba90-4992-8687-2a13a180f4cd
Secret key header:
a6528f8538490780ddc57640ce0f73c4304970e6faad81f4de452b929bcdf011

1. Overview

Raast is Pakistan's national instant payment network operated by the State Bank of Pakistan (SBP). It enables real-time, interoperable fund transfers between bank accounts and mobile wallets across all participating financial institutions in Pakistan. Safepay Raast exposes an aggregator-grade API layer on top of the Raast network, providing Simpaisa with the infrastructure to initiate payments, manage merchants, issue payouts, and receive settlement notifications through a single set of credentials.

Simpaisa operates as the **Aggregator** in this model. Under the aggregator account, Simpaisa onboards its own downstream clients as **Aggregator Merchants**. Each Aggregator Merchant must be linked to a **Raast Merchant**, which is the Raast-network identity registered with SBP

through Safepay. All payments, payouts, and QR codes are ultimately attributed to a Raast Merchant, making this linkage a prerequisite for live transactions.

The integration supports four payment flows: **RTP Now** (immediate customer-approved Request To Pay), **RTP Later** (deferred Request To Pay with a configurable window), **Dynamic QR** (per-transaction, amount-embedded QR codes), and **Static QR** (reusable, open-amount QR codes for countertop or invoice use). All payment outcomes are **asynchronous** — webhooks are the primary source of truth for final status, and polling is a secondary fallback.

Note: This document has multiple features and payment method integrations.

1. RTP Now
2. RTP Later
3. Dynamic QR
4. Static QR

As the priority is RTP Now, we still recommend integrating RTP Now and Later together as they use the same functionality and API Specs.

Integration Path

1. **Request aggregator access** — Contact Safepay to receive your `aggregator_id` and `secret_key`.
2. **Store your secret key** — Save `secret_key` in a server-side secret manager. Never expose it client-side.
3. **Verify credentials** — Call `GET /v1/aggregators/{{raast-aggregator-id}}` to confirm your credentials are active before building any payment flows.
4. **Create and link merchants** — Create Aggregator Merchants via the Merchants API. Link each to a Raast Merchant. If a Raast Merchant does not yet exist, a KYC process with Safepay is required (see Section 12).
5. **Choose a payment flow** — Implement RTP Now, RTP Later, Dynamic QR, or Static QR based on your use case.
6. **Subscribe to webhooks** — Register a webhook endpoint to receive payment, settlement, and refund status events in real time. Without registering to a webhook and events you will not receive respective webhooks.

---

2. Environments & Base URLs

Environment	Base URL	Notes

Production	`https://api.getsafepay.com/raastwire`	Live traffic. Real money movement.

> All endpoint paths in this document are relative to the appropriate base URL. Replace `{{baseUrl}}` with the correct environment URL in all cURL examples. Please note that the sandbox environment exists but does not work properly. So safepay has recommended to use the live environment only.

---

3. Authentication

Every API request must include the aggregator secret key as a request header:

```
text
X-SFPY-AGGREGATOR-SECRET-KEY: {{secret_key}}
```

The `secret_key` is the master credential issued by Safepay when your aggregator account is created. It authenticates all operations — payments, payouts, merchant management, and webhooks — under your aggregator identity. This secret key can be changed using Access Key Management. Refer to Section 6.

> **Security requirement:** Store `secret_key` exclusively in a server-side secret manager (e.g., AWS Secrets Manager, HashiCorp Vault). Never embed it in frontend code, mobile apps, or version control. Exposure of this key compromises your entire aggregator account.

> **M** = Mandatory | **O** = Optional | **C** = Conditional

---

4. Get Aggregator (Credential Verification)

This is the **first API call** you should make after receiving credentials. Call this endpoint to verify that your `aggregator_id` and `secret_key` are valid and that your aggregator account is active before building any payment flows.

Get Aggregator

Description: Retrieves full metadata for your aggregator account, including active access keys, webhook configuration, and vault status. Use this call to confirm credentials are correct before proceeding with merchant onboarding or payment initiation.

Method: `GET`

Endpoint: `/v1/aggregators/{{raast-aggregator-id}}`

Authentication: Header — `X-SFPY-AGGREGATOR-SECRET-KEY: {{secret_key}}`

Request Parameters

Parameter	Type	Data Type	Mandatory	Sample Value	Description
X-SFPY-AGGREGATOR-SECRET-KEY	Header	String	M	`{{secret_key}}`	Aggregator secret key for authentication
raast-aggregator-id	Path	String	M	`agg_2288490a-2176-4de5-b373-0ffb6f8e2e6e`	Unique aggregator token issued by Safepay

---

Sample Request (cURL)

```

bash
curl --request GET \
  --url "{{baseUrl}}/v1/aggregators/{{raast-aggregator-id}}" \
  --header "X-SFPY-AGGREGATOR-SECRET-KEY: {{secret_key}}"

```

Response Parameters

Parameter	Data Type	Mandatory	Sample Value	Description
`api_version`	String	M	`"v1"`	API version
`data.id`	String	M	`"2"`	Internal numeric ID of the aggregator
`data.token`	String	M	`"agg_2288490a-..."`	Aggregator token; this is your `raast-aggregator-id`
`data.partner_id`	String	M	`"partner_ed4f0480-..."`	Partner identifier
`data.bank_account_id`	String	M	`"acc_c03a57cb-..."`	Linked bank account identifier
`data.name`	String	M	`"Safepay"`	Display name of the aggregator
`data.is_active`	Boolean	M	`true`	Whether the aggregator account is active
`data.vault_enabled`	Boolean	M	`true`	Whether vault tokenization is enabled for this aggregator

`data.webhook_url`	String	O	`"www.google.com"`	Configured webhook delivery URL
`data.webhook_secret`	String	O	`"OC5FFVV72GRA8Y5AFE6JDE4YFYS4QF"`	Secret used to verify incoming webhook signatures
`data.Keys[]`	Array	M	—	Array of access key objects associated with this aggregator
`data.Keys[].id`	String	M	`"2"`	Internal ID of the access key
`data.Keys[].token`	String	M	`"ak_fa66e6f5-..."`	Access key token identifier
`data.Keys[].key_id`	String	M	`"key_452442dc-..."`	Unique key ID
`data.Keys[].aggregator_id`	String	M	`"agg_2288490a-..."`	Parent aggregator token
`data.Keys[].is_active`	Boolean	M	`true`	Whether this key is active
`data.Keys[].name`	String	M	`"Staging key"`	Human-readable label for the key

`data.Keys[].secret`	String	M	`"18108aad..."`	Key secret value
`data.Keys[].created_at`	String	M	`"2025-03-13T09:12:51Z"`	Key creation timestamp (ISO 8601)
`data.Keys[].updated_at`	String	M	`"2025-03-13T09:12:51Z"`	Key last-updated timestamp (ISO 8601)
`data.created_at`	String	M	`"2025-03-13T09:12:34Z"`	Aggregator account creation timestamp
`data.updated_at`	String	M	`"2025-05-23T09:58:39Z"`	Aggregator last-updated timestamp

Sample Response

Success (HTTP 201):

```
json
{
    "api_version": "v1",
    "data": {
        "id": "2",
        "token": "agg_2288490a-2176-4de5-b373-0ffb6f8e2e6e",
        "partner_id": "partner_ed4f0480-36c8-4a5a-8cd4-381bc4d83f2c",
        "bank_account_id": "acc_c03a57cb-a616-42a7-b1dc-de473217a558",
        "name": "Safepay",
        "is_active": true,
        "vault_enabled": true,
        "webhook_url": "www.google.com",
        "webhook_secret": "OC5FFVV72GRA8Y5AFE6JDE4YFYS4QF",
        "Keys": [
            {
                "id": "2",
                "token": "ak_fa66e6f5-cd88-4d8a-8fe7-0eb1898d20a3",
                "key_id": "key_452442dc-d6b3-4cbb-bcc6-8781ef2d3498",
                "aggregator_id": "agg_2288490a-2176-4de5-b373-
0ffb6f8e2e6e",
                "is_active": true,
                "name": "Staging key",
                "secret":
"18108aad30da06075e6f02fab3166926827538363842185faf6e3e694ea5d481",
```

```
                "created_at": "2025-03-13T09:12:51Z",
                "updated_at": "2025-03-13T09:12:51Z"
            },
            {
                "id": "4",
                "token": "ak_da8f1c10-346a-4be8-895a-b6187a9a9662",
                "key_id": "key_5430e06f-cefb-4393-8b7a-489c8d7c1128",
                "aggregator_id": "agg_2288490a-2176-4de5-b373-
0ffb6f8e2e6e",
                "is_active": true,
                "name": "prod key",
                "secret":
"3a7c97a7cb15cb90a46503ab577298e3619ae67e5b885cdd34efc9f24a3816ee",
                "created_at": "2025-05-23T10:02:26Z",
                "updated_at": "2025-05-23T10:05:34Z"
            }
        ],
        "created_at": "2025-03-13T09:12:34Z",
        "updated_at": "2025-05-23T09:58:39Z"
    }
}

```


Error (HTTP 404):


```

json
{
    "code": "error.resource_not_found",
    "message": "code=404, message=Not Found"
}

```


---

5. Access Keys Management

Access keys are named credentials scoped to your aggregator account. They support rotation for key hygiene and can be deactivated without revoking the master `secret_key`. Use these APIs to audit active keys, update metadata, or rotate secrets in production.

5a. List Access Keys

Description: Fetches a paginated list of access keys associated with the aggregator. Supports filtering by active status and name. Use this to audit which keys are active and to discover key tokens needed for update or rotate operations.

Method: `GET`

Endpoint: `/v1/aggregators/{{raast-aggregator-id}}/keys`

Authentication: Header — `X-SFPY-AGGREGATOR-SECRET-KEY: {{secret_key}}`

Request Parameters

Parameter	Type	Data Type	Mandatory	Sample Value	Description
X-SFPY-AGGREGATOR-SECRET-KEY	Header	String	M	`{{secret_key}}`	Aggregator secret key for authentication
raast-aggregator-id	Path	String	M	`agg_2288490a-...`	Aggregator token
limit	Query	Integer	O	`10`	Maximum number of records to return. Default: 10
offset	Query	Integer	O	`0`	Number of records to skip for pagination. Default: 0
names	Query	String	O	`"prod key"`	Filter results by access key name
is_active	Query	Boolean	O	`true`	Filter by active state (`true` or `false`)

Sample Request (cURL)

```
bash
curl --request GET \
```

```
  --url "{{baseUrl}}/v1/aggregators/{{raast-aggregator-id}}/keys?limit=10&offset=0&is_active=true" \
  --header "X-SFPY-AGGREGATOR-SECRET-KEY: {{secret_key}}"

```

Response Parameters

Parameter	Data Type	Mandatory	Sample Value	Description
`api_version`	String	M	`"v1"`	API version
`data.keys[]`	Array	M	—	Array of access key objects
`data.keys[].id`	String	M	`"2"`	Internal numeric ID
`data.keys[].token`	String	M	`"ak_fa66e6f5-..."`	Access key token; use as `{{access-key-id}}` in update/rotate calls
`data.keys[].key_id`	String	M	`"key_452442dc-..."`	Unique key identifier
`data.keys[].aggregator_id`	String	M	`"agg_2288490a-..."`	Parent aggregator token
`data.keys[].is_active`	Boolean	M	`true`	Whether this key is currently active
`data.keys[].name`	String	M	`"Staging key"`	Human-readable label

`data.keys[].secret`	String	M	`"18108aad..."`	Key secret value
`data.keys[].created_at`	String	M	`"2025-03-13T09:12:51Z"`	Creation timestamp
`data.keys[].updated_at`	String	M	`"2025-03-13T09:12:51Z"`	Last-updated timestamp
`data.count`	String	C	`"2"`	Total count of matching keys (present when filters are applied)

Sample Response

Success (HTTP 200 — Base):

```
json
{
    "api_version": "v1",
    "data": {
        "keys": [
            {
                "id": "2",
                "token": "ak_fa66e6f5-cd88-4d8a-8fe7-0eb1898d20a3",
                "key_id": "key_452442dc-d6b3-4cbb-bcc6-8781ef2d3498",
                "aggregator_id": "agg_2288490a-2176-4de5-b373-0ffb6f8e2e6e",
                "is_active": true,
                "name": "Staging key",
                "secret": "18108aad30da06075e6f02fab3166926827538363842185faf6e3e694ea5d481",
                "created_at": "2025-03-13T09:12:51Z",
                "updated_at": "2025-03-13T09:12:51Z"
            },
            {
                "id": "4",
                "token": "ak_da8f1c10-346a-4be8-895a-b6187a9a9662",
                "key_id": "key_5430e06f-cefb-4393-8b7a-489c8d7c1128",
                "aggregator_id": "agg_2288490a-2176-4de5-b373-0ffb6f8e2e6e",
                "is_active": true,
                "name": "prod key",
                "secret": "3a7c97a7cb15cb90a46503ab577298e3619ae67e5b885cdd34efc9f24a3816ee",
                "created_at": "2025-05-23T10:02:26Z",
                "updated_at": "2025-05-23T10:05:34Z"
```

```
            }
        ]
    }
}

```

Success (HTTP 200 — With `is_active` filter and count):

```

json
{
    "api_version": "v1",
    "data": {
        "keys": [
            {
                "id": "4",
                "token": "ak_da8f1c10-346a-4be8-895a-b6187a9a9662",
                "key_id": "key_28d69595-4ae4-4645-b9d8-300fa5211845",
                "aggregator_id": "agg_2288490a-2176-4de5-b373-
0ffb6f8e2e6e",
                "is_active": true,
                "name": "prod key",
                "secret":
"23052bb4602992643b8cd3aaee10a1054a5b071a2ecd3bf2e39ba5072ad3f4cd",
                "created_at": "2025-05-23T10:02:26Z",
                "updated_at": "2025-07-02T09:16:55Z"
            }
        ],
        "count": "2"
    }
}

```

Success (HTTP 200 — With `names` filter):

```

json
{
    "api_version": "v1",
    "data": {
        "keys": [
            {
                "id": "4",
                "token": "ak_da8f1c10-346a-4be8-895a-b6187a9a9662",
                "key_id": "key_28d69595-4ae4-4645-b9d8-300fa5211845",
                "aggregator_id": "agg_2288490a-2176-4de5-b373-
0ffb6f8e2e6e",
                "is_active": true,
                "name": "prod key",
                "secret":
"23052bb4602992643b8cd3aaee10a1054a5b071a2ecd3bf2e39ba5072ad3f4cd",
                "created_at": "2025-05-23T10:02:26Z",
                "updated_at": "2025-07-02T09:16:55Z"
            }
        ],
        "count": "1"
    }
}

```

Error (HTTP 401):

```

json
```

```
{
    "api_version": "v1",
    "error": {
        "code": "error.unauthorized_access",
        "message": "aggregator not found"
    }
}

```

5b. Update Access Key

Description: Updates an existing access key's `name` and/or `is_active` status. Use this to label keys meaningfully or to deactivate a compromised key without deleting it. Only `is_active` and `name` are updatable fields.

Method: `PUT`

Endpoint: `/v1/aggregators/{{raast-aggregator-id}}/keys/{{access-key-id}}`

Authentication: Header — `X-SFPY-AGGREGATOR-SECRET-KEY: {{secret_key}}`

Request Parameters

Parameter	Type	Data Type	Mandatory	Sample Value	Description
X-SFPY-AGGREGATOR-SECRET-KEY	Header	String	M	`{{secret_key}}`	Aggregator secret key
raast-aggregator-id	Path	String	M	`agg_2288490a-...`	Aggregator token
access-key-id	Path	String	M	`ak_da8f1c10-...`	Token of the access key to update
is_active	Body	Boolean	O	`true`	Set to `false` to

					deactivate the key
name	Body	String	O	`"prod key"`	New display name for the key

Sample Request (cURL)

```
bash
curl --request PUT \
  --url "{{baseUrl}}/v1/aggregators/{{raast-aggregator-id}}/keys/{{access-key-id}}" \
  --header "X-SFPY-AGGREGATOR-SECRET-KEY: {{secret_key}}" \
  --header "Content-Type: application/json" \
  --data '{
    "is_active": true,
    "name": "prod key"
}'
```

Response Parameters

Parameter	Data Type	Mandatory	Sample Value	Description
`api_version`	String	M	`"v1"`	API version
`data.message`	String	M	`"success"`	Confirmation of update

Sample Response

Success (HTTP 200):

```
json
{
    "api_version": "v1",
    "data": {
        "message": "success"
    }
}
```

5c. Rotate Access Key

Description: Rotates an existing access key by generating a new secret while invalidating the old one. The key token (`ak_...`) remains the same; only the `secret` value changes. Use this as part of regular key hygiene or after a suspected secret exposure.

Method: `PUT`

Endpoint: `/v1/aggregators/{{raast-aggregator-id}}/keys/{{access-key-id}}/rotate`

Authentication: Header — `X-SFPY-AGGREGATOR-SECRET-KEY: {{secret_key}}`

Request Parameters

Parameter	Type	Data Type	Mandatory	Sample Value	Description
X-SFPY-AGGREGATOR-SECRET-KEY	Header	String	M	`{{secret_key}}`	Aggregator secret key
raast-aggregator-id	Path	String	M	`agg_2288490a-...`	Aggregator token
access-key-id	Path	String	M	`ak_da8f1c10-...`	Token of the access key to rotate

---

Sample Request (cURL)

```

bash
curl --request PUT \
  --url "{{baseUrl}}/v1/aggregators/{{raast-aggregator-id}}/keys/{{access-key-id}}/rotate" \
  --header "X-SFPY-AGGREGATOR-SECRET-KEY: {{secret_key}}"

```

Response Parameters

Parameter	Data Type	Mandatory	Sample Value	Description
`api_version`	String	M	`"v1"`	API version
`data.id`	String	M	`"4"`	Internal key ID
`data.token`	String	M	`"ak_da8f1c10-..."`	Access key token (unchanged after rotation)
`data.key_id`	String	M	`"key_5430e06f-..."`	Unique key identifier
`data.aggregator_id`	String	M	`"agg_2288490a-..."`	Parent aggregator
`data.is_active`	Boolean	M	`true`	Key active status
`data.name`	String	M	`"prod key"`	Key label
`data.secret`	String	M	`"3a7c97a7..."`	**New** rotated secret value — store this immediately
`data.created_at`	String	M	`"2025-05-23T10:02:26Z"`	Original creation timestamp
`data.updated_at`	String	M	`"2025-05-23T10:05:15Z"`	Rotation timestamp

Sample Response

Success (HTTP 200):

```
json
{
    "api_version": "v1",
    "data": {
        "id": "4",
        "token": "ak_da8f1c10-346a-4be8-895a-b6187a9a9662",
        "key_id": "key_5430e06f-cefb-4393-8b7a-489c8d7c1128",
        "aggregator_id": "agg_2288490a-2176-4de5-b373-0ffb6f8e2e6e",
        "is_active": true,
        "name": "prod key",
        "secret":
"3a7c97a7cb15cb90a46503ab577298e3619ae67e5b885cdd34efc9f24a3816ee",
        "created_at": "2025-05-23T10:02:26Z",
        "updated_at": "2025-05-23T10:05:15Z"
    }
}
```

> **Important:** The new `secret` value is returned only once in the rotation response. Store it in your secret manager immediately. Subsequent calls to List Access Keys will still show the key, but the plain-text secret will not be re-exposed.

---

6. Utilities

Use these utility endpoints to validate debtor or beneficiary account details **before** initiating any RTP payment or payout. Validating account details upfront reduces failed transactions and avoids unnecessary charges.

6a. Title Fetch (by IBAN)

Description: Returns the officially registered account title for a Raast IBAN by querying the Central Account Service (CAS). Use this before initiating an RTP or payout to confirm the destination account holder name matches the expected beneficiary. This API will only title-fetch for the IBAN to confirm the validity of the account entered.

Method: `GET`

Endpoint: `/v1/aggregators/{{raast-aggregator-id}}/title-fetch?iban={{iban}}`

Authentication: Header — `X-SFPY-AGGREGATOR-SECRET-KEY: {{secret_key}}`

Request Parameters

Parameter	Type	Data Type	Mandatory	Sample Value	Description
X-SFPY-AGGREGATOR-SECRET-KEY	Header	String	M	`{{secret_key}}`	Aggregator secret key
raast-aggregator-id	Path	String	M	`agg_2288490a-...`	Aggregator token
iban	Query	String	M	`PK89SCBL0000001234651601`	Full Pakistani IBAN to resolve

Sample Request (cURL)

```
bash
curl --request GET \
  --url "{{baseUrl}}/v1/aggregators/{{raast-aggregator-id}}/title-fetch?iban=PK89SCBL0000001234651601" \
  --header "X-SFPY-AGGREGATOR-SECRET-KEY: {{secret_key}}"
```

Response Parameters

Parameter	Data Type	Mandatory	Sample Value	Description
`api_version`	String	M	`"v1"`	API version

`data.account_title`	String	M	`"Muhammad Ali"`	Registered account holder name from CAS
`data.account_type`	String	M	`"Account"`	Type of bank account
`data.iban`	String	M	`"PK89SCBL..."`	Resolved IBAN

Sample Response

Success (HTTP 200):

```
json
{
    "api_version": "v1",
    "data": {
        "iban": "PK89SCBL0000001234651601",
        "account_title": "Muhammad Ali",
        "account_type": "Account"
    }
}
```

6b. Account Info (by IBAN)

Description: Given an IBAN, this endpoint retrieves account information and — when `with_title_fetch=true` — additionally queries CAS to return the registered account title and account type. Use this when the debtor or beneficiary is identified by a direct IBAN.

Method: `GET`
Endpoint: `/v1/aggregators/{{raast-aggregator-id}}/account-info?iban={{iban}}`
Authentication: Header — `X-SFPY-AGGREGATOR-SECRET-KEY: {{secret_key}}`

Request Parameters

Type	Parameter	Data Type	Mandatory	Sample Value	Description

Header	X-SFPY-AGGREGATOR-SECRET-KEY	String	M	`{{secret_key}}`	Aggregator secret key
Path	raast-aggregator-id	String	M	`agg_2288490a-...`	Aggregator token
Query	iban	String	M	`PK91ALFH0005001007079203`	Target account IBAN
Query	with_title_fetch	Boolean	O	`true`	If true, also fetches account title and type from CAS. Default: `false`

Sample Request (cURL)

```
bash
curl --location --globoff 'https://api.getsafepay.com/raastwire{{raast-wire}}/v1/aggregators/{{raast-aggregator-id}}/account-info?iban=PK91ALFH0005001007079203&with_title_fetch=true' \
  --header 'X-SFPY-AGGREGATOR-SECRET-KEY: {{X-SFPY-AGGREGATOR-SECRET-KEY}}'
```

Response Parameters

Parameter	Data Type	Mandatory	Sample Value	Description
api_version	String	M	`"v1"`	API version

data.iban	String	M	`"PK91ALFH000500100 7079203"`	Resolved IBAN
data.type	String	M	`"CAS_CONS T_UNSPECI FIED"`	Alias type descriptor
data.currenc y	String	M	`"PKR"`	Account currency
data.servicer _member_id	String	M	`"ALFHPKKA "`	Servicer member identifier
data.name	String	M	`"MIRZA"`	Account holder name
data.is_defau lt	Boolean	M	`false`	Whether this is the default alias
data.dba	String	O	`""`	Doing-business-as name
data.mcc	String	O	`""`	Merchant category code
data.account _title	String	C	`"Title-1 100707920 3"`	Account holder title (present when `with_titl e_fetch=t rue`)
data.account _type	String	C	`"Account"`	Account type (present when `with_titl`

				`e_fetch=true`)

Sample Response

Success (HTTP 200):

```
json
{
    "api_version": "v1",
    "data": {
        "iban": "PK91ALFH0005001007079203",
        "type": "CAS_CONST_UNSPECIFIED",
        "currency": "PKR",
        "servicer_member_id": "ALFHPKKA",
        "name": "MIRZA",
        "is_default": false,
        "dba": "",
        "mcc": "",
        "account_title": "Title-1 1007079203",
        "account_type": "Account"
    }
}
```

6c. Account Info (by Alias — Raast ID)

Description: Given a Raast alias (Mobile number), this endpoint resolves the associated IBAN and — when `with_title_fetch=true` — additionally queries Safepay to return the registered account title and account type. Use this when the debtor or beneficiary is identified by a mobile number alias rather than a direct IBAN. Through this Title Fetch of a Raast ID will also be done.

Method: `GET`
Endpoint: `/v1/aggregators/{{raast-aggregator-id}}/account-info?alias_type={{alias_type}}&alias_value={{alias_value}}`
Authentication: Header — `X-SFPY-AGGREGATOR-SECRET-KEY: {{secret_key}}`

Request Parameters

Parameter	Type	Data Type	Mandatory	Sample Value	Description
X-SFPY-AGGREGATOR-	Header	String	M	`{{secret_key}}`	Aggregator secret key

SECRET-KEY					
raast-aggregator-id	Path	String	M	`agg_2288490a-...`	Aggregator token
alias_type	Query	String	M	`MOBILE`	Alias type (e.g. MOBILE)
alias_value	Query	String	M	`"03472840476"`	Mobile number linked to the Raast alias
with_title_fetch	Query	Boolean	O	`true`	If true, also fetches account title and type from CAS. Default: `false`

Sample Request (cURL)

```
bash
curl --location --globoff 'https://api.getsafepay.com/raastwire{{raast-wire}}/v1/aggregators/{{raast-aggregator-id}}/account-info?with_title_fetch=true&alias_type=MOBILE&alias_value=03472840476' \
  --header 'X-SFPY-AGGREGATOR-SECRET-KEY: {{X-SFPY-AGGREGATOR-SECRET-KEY}}'
```

Response Parameters

Parameter	Data Type	Mandatory	Sample Value	Description
api_version	String	M	`"v1"`	API version

data.iban	String	M	`"PK74SONE000022000 8098833"`	Resolved IBAN for the alias
data.type	String	M	`"CAS_CONS T_UNSPECI FIED"`	Alias type descriptor
data.currenc y	String	M	`"PKR"`	Account currency
data.servicer _member_id	String	M	`"SONEPKKA "`	Servicer member identifier
data.name	String	M	`"ZAIN BIN OMER"`	Account holder name linked to this alias
data.is_defau lt	Boolean	M	`true`	Whether this is the default alias
data.dba	String	O	`""`	Doing-business-as name
data.mcc	String	O	`""`	Merchant category code
data.account _title	String	C	`"ZAIN BIN OMER"`	Account holder title (present when `with_titl e_fetch=t rue`)
data.account _type	String	C	`"Account"`	Account type (present

				when `with_title_fetch=true`)

Sample Response

Success (HTTP 200):

```
json
{
    "api_version": "v1",
    "data": {
        "iban": "PK74SONE0000220008098833",
        "type": "CAS_CONST_UNSPECIFIED",
        "currency": "PKR",
        "servicer_member_id": "SONEPKKA",
        "name": "ZAIN BIN OMER",
        "is_default": true,
        "dba": "",
        "mcc": "",
        "account_title": "ZAIN BIN OMER",
        "account_type": "Account"
    }
}
```

7. Payments

All payments are asynchronous. The POST creation endpoints return an initial `P_INITIATED` status and a `token` (payment token). The final outcome — success, failure, or rejection — arrives via webhook. Polling the GET endpoint is a supported fallback.

> **Debtor identifier rule:** For all RTP payments, supply **exactly one** of `debitor_iban`, `debitor_raast_id`, `debitor_vault_token`, or `debitor_alias` in each request. Sending more than one will result in a validation error.

7a. RTP Now

RTP Now sends an immediate Request To Pay to the customer's banking app. The customer receives a push notification and must approve or reject in real time. Use this flow for checkout flows, point-of-sale collections, and any scenario where instant customer action is expected.

Flow Summary

1. Retrieve the `aggregator_merchant_identifier` from the merchant creation response.
2. Validate the debtor account using Title Fetch (Section 7) to confirm account details.
3. Submit a `POST /v1/aggregators/{{raast-aggregator-id}}/payments` request with `type: "RTP_NOW"` and the debtor identifier.
4. The customer's banking app delivers a push notification; the customer approves or rejects.
5. Monitor the outcome via webhook (primary) or poll the GET payment endpoint (secondary).

Create RTP Now Payment

Description: Initiates an immediate Request To Pay. The debtor receives a bank notification and must approve within the `expiry_in_minutes` window. Returns a payment token and initial status of `P_INITIATED`.

Method: `POST`

Endpoint: `/v1/aggregators/{{raast-aggregator-id}}/payments`

Authentication: Header — `X-SFPY-AGGREGATOR-SECRET-KEY: {{secret_key}}`

Request Parameters

Type	Parameter	Data Type	Mandatory	Sample Value	Description
Header	X-SFPY-AGGREGATOR-SECRET-KEY	String	M	`{{secret_key}}`	Aggregator secret key
Body	request_id	String (UUID)	M	`"3fa85f64-5717-..."`	UUID enforcing idempotency. Reuse only when retrying the

					identical payload.
Body	amount	Integer	M	`34000`	Amount in paisa. `34000` = PKR 340.00
Body	aggregator_merchant_identifier	String	M	`"am_749fe6ce-..."`	Token returned when the merchant was created (`data.token`)
Body	order_id	String	M	`"order_001"`	Merchant order reference echoed in responses and webhooks
Body	type	String	M	`"RTP_NOW"`	Must be `RTP_NOW` for this flow
Body	expiry_in_minutes	Integer	O	`100`	Request validity window in minutes. Range: 1–180. Default: 180

Body	debitor_iban	String	C	`"PK89SCBL00000123465 1601"`	Debtor IBAN. Provide exactly one debtor identifier.
Body	debitor_raast_id	String	C	`"032022 96111"`	Debtor Raast ID (e.g., mobile number). Provide exactly one debtor identifier.
Body	debitor_vault_token	String	C	`"lock_4 41d2a6f -..."`	Vault-stored payment method token. Provide exactly one debtor identifier.
Body	debitor_alias	String	C	`"541661 038"`	Debtor alias. Provide exactly one debtor identifier.

Sample Request (cURL)

```
bash
curl --request POST \
```

```
  --url "{{baseUrl}}/v1/aggregators/{{raast-aggregator-id}}/payments" \
  --header "X-SFPY-AGGREGATOR-SECRET-KEY: {{secret_key}}" \
  --header "Content-Type: application/json" \
  --data '{
    "request_id": "{{$guid}}",
    "amount": 34000,
    "debitor_iban": "PK89SCBL0000001234651601",
    "aggregator_merchant_identifier": "am_749fe6ce-91f9-4f6a-9df2-ec6ae9f75a30",
    "order_id": "test_nowExpiry",
    "type": "RTP_NOW",
    "expiry_in_minutes": 100
}'

```

Response Parameters

Parameter	Data Type	Mandatory	Sample Value	Description
`api_version`	String	M	`"v1"`	API version
`data.message`	String	M	`"success"`	Creation confirmation
`data.token`	String	M	`"pm_68cd947a-..."`	Payment token used to poll status
`data.msg_id`	String	M	`"32933KC22V3CKQ71..."`	Raast network message ID
`data.trace_reference`	String	M	`"cb08be1d-..."`	Internal trace reference for reconciliation
`data.uetr`	String	M	`"17467849-b891-..."`	Unique end-to-end transaction reference
`data.status`	String	M	`"P_INITIATED"`	Initial payment status

`data.order_id`	String	M	`"test_nowExpiry"`	Echoed merchant order reference
`data.created_at`	String	M	`"2025-07-29T05:30:38Z"`	Payment creation timestamp

Sample Response

Success (HTTP 200):

```
json
{
    "api_version": "v1",
    "data": {
        "message": "success",
        "token": "pm_68cd947a-03c7-40b1-88f8-bd7be978da98",
        "msg_id": "32933KC22V3CKQ71USVKXSZAEA2Q8AA3ZN2",
        "trace_reference": "cb08be1d-4728-4433-85a8-01bb8ae3061b",
        "uetr": "17467849-b891-41f0-a1ce-cae8f97484c6",
        "status": "P_INITIATED",
        "order_id": "test_nowExpiry",
        "created_at": "2025-07-29T05:30:38.216536926Z"
    }
}
```

Error (HTTP 500):

```
json
{
    "api_version": "v1",
    "error": {
        "code": "error.internal_server_error",
        "message": "rpc error: code = Internal desc = something went
wrong"
    }
}
```

Get Payment (Status Poll)

Description: Retrieves the current status and full detail of a previously created payment. Use this endpoint for fallback polling when a webhook has not been received, or to populate dashboard views.

Method: `GET`

Endpoint: `/v1/aggregators/{{raast-aggregator-id}}/payments/{{payment_token}}`

Authentication: Header — `X-SFPY-AGGREGATOR-SECRET-KEY: {{secret_key}}`

Request Parameters

Type	Parameter	Data Type	Mandatory	Sample Value	Description
Header	X-SFPY-AGGREGATOR-SECRET-KEY	String	M	`{{secret_key}}`	Aggregator secret key
Path	raast-aggregator-id	String	M	`agg_2288490a-...`	Aggregator token
Path	payment_token	String	M	`pm_68cd947a-...`	Payment token from the create payment response

---

Sample Request (cURL)

```

bash
curl --request GET \
  --url "{{baseUrl}}/v1/aggregators/{{raast-aggregator-id}}/payments/{{payment_token}}" \
  --header "X-SFPY-AGGREGATOR-SECRET-KEY: {{secret_key}}"

```

Sample Response

Success (HTTP 200):

```

```

```
json
{
    "api_version": "v1",
    "data": {
        "id": "641",
        "token": "pm_b4a30cc5-f0ec-4c61-8fe3-3ece42196c81",
        "aggregator_merchant_id": "am_d7fd05ea-8613-4038-afd7-
4275f29110f5",
        "qr_code_id": "",
        "order_id": "NewPayLater1",
        "type": "RTP_NOW",
        "amount": "34000",
        "status": "P_CAPTURED",
        "uetr": "f4bbe7db-9f5f-4fe3-92e1-2de04f492713",
        "trace_reference": "0c843220-2813-4573-83e6-180ad99c2933",
        "msg_id": "ZCPC3GS3LJNPMXQBN7G3LXU7WDJJ6PB2JVK",
        "instr_id": "fabce92286f14ab3852aed0e3e5d28ec",
        "end_to_end_id": "f4bbe7db9f5f4fe392e12de04f492713",
        "pmt_inf_id": "AG9G53RMNQRLFKX1AZCT6WG3GD9SZ6O8OQ7",
        "request_id": "d4d5d335-af3e-4de7-9724-26fad5026366",
        "msg_created_at": "2025-06-26 15:58:25.",
        "expires_at": "2025-07-03T10:58:26Z",
        "created_at": "2025-06-26T10:58:26Z",
        "updated_at": "2025-06-27T09:49:48Z",
        "txn_logs": [],
        "debitor": null,
        "qr_code": null,
        "aggregator_merchant": null,
        "refunds": [],
        "settlement": null
    }
}

```

Webhook Events to Expect — RTP Now

- `payment.created` — Payment request was created
- `payment.pending_authorization` — Awaiting customer authorization
- `payment.authorized` — Customer approved; funds on hold
- `payment.completed` — Payment captured successfully
- `payment.settled` — Funds settled to merchant
- `payment.rejected` — Customer rejected the request
- `payment.failed` — Processing failure
- `payment.voided` — Authorization voided
- `payment.reversed` — Reversed after completion
- `payment.refunded` — Full refund issued (if applicable)
- `payment.refund_partial` — Partial refund issued (if applicable)

Troubleshooting — RTP Now

- **Invalid debtor details** — Run Title Fetch (Section 6) before submitting the RTP to validate the IBAN or alias.

- **Expired request** — Regenerate the payment with a fresh `request_id` and updated `expiry_in_minutes`.
- **Duplicate payload error** — Reusing a `request_id` with different body parameters triggers an idempotency error. To retry, send the exact same payload.
- **Internal server error (500)** — Confirm `amount` is an integer (not a string), and that the `aggregator_merchant_identifier` is valid and active.

7b. RTP Later

RTP Later creates a deferred Request To Pay. The customer may approve or reject hours or days after the request is submitted. Use this flow for invoice settlements, subscription renewals, and scenarios where immediate customer presence is not guaranteed.

Flow Summary

1. Retrieve the `aggregator_merchant_identifier` from the merchant creation response.
2. Validate the debtor account using Title Fetch (Section 6).
3. Submit `POST /v1/aggregators/{{raast-aggregator-id}}/payments` with `type: "RTP_LATER"` and an `expiry_in_days` value.
4. The customer's banking app may deliver the notification immediately or the customer may see it later.
5. Monitor the outcome via webhook (primary) or poll the GET payment endpoint (secondary).

Create RTP Later Payment

Description: Schedules a Request To Pay with a configurable day-level expiry window. Suitable for flows where customers approve payments asynchronously. Returns a payment token and initial status of `P_INITIATED`.

Method: `POST`

Endpoint: `/v1/aggregators/{{raast-aggregator-id}}/payments`

Authentication: Header — `X-SFPY-AGGREGATOR-SECRET-KEY: {{secret_key}}`

Request Parameters

Type	Parameter	Data Type	Mandatory	Sample Value	Description

Header	X-SFPY-AGGREGATOR-SECRET-KEY	String	M	`{{secret_key}}`	Aggregator secret key
Body	request_id	String (UUID)	M	`"3fa85f64-5717-..."`	UUID for idempotency. Reuse only when retrying the same payload.
Body	amount	Integer	M	`5000`	Amount in paisa. `5000` = PKR 50.00
Body	aggregator_merchant_identifier	String	M	`"am_749fe6ce-..."`	Token from merchant creation (`data.token`)
Body	order_id	String	M	`"order_001"`	Merchant order reference
Body	type	String	M	`"RTP_LATER"`	Must be `RTP_LATER` for this flow
Body	expiry_in_days	Integer	O	`12`	Approval window in days. Range: 1–40.

					Default: 39 days
Body	debitor_iban	String	C	`"PK89SCBL000001234651601"`	Debtor IBAN. Provide exactly one debtor identifier.
Body	debitor_raast_id	String	C	`"03202296111"`	Debtor Raast ID. Provide exactly one debtor identifier.
Body	debitor_vault_token	String	C	`"lock_441d2a6f-..."`	Vault-stored payment method. Provide exactly one debtor identifier.
Body	debitor_alias	String	C	`"541661038"`	Debtor alias. Provide exactly one debtor identifier.

Sample Request (cURL)

```
bash
curl --request POST \
  --url "{{baseUrl}}/v1/aggregators/{{raast-aggregator-id}}/payments"
\
  --header "X-SFPY-AGGREGATOR-SECRET-KEY: {{secret_key}}" \
  --header "Content-Type: application/json" \
```

```
  --data '{
    "request_id": "{{$guid}}",
    "amount": 5000,
    "debitor_iban": "PK89SCBL0000001234651601",
    "aggregator_merchant_identifier": "am_749fe6ce-91f9-4f6a-9df2-
ec6ae9f75a30",
    "order_id": "test_iban_later",
    "type": "RTP_LATER",
    "expiry_in_days": 12
}'

```

Sample Response

Success (HTTP 200):

```

json
{
    "api_version": "v1",
    "data": {
        "message": "success",
        "token": "pm_b53a233e-ab73-4486-a3c9-789caa411f59",
        "msg_id": "O0KY9IBH7CG0FENRBH5TOC8D8C00FD7P9FB",
        "trace_reference": "0e0d4c14-017e-43e7-bc6e-084906d6acda",
        "uetr": "3b05735f-2efd-4eba-8e97-7011bd88d2b8",
        "status": "P_INITIATED",
        "order_id": "test_iban_later",
        "created_at": "2025-07-29T05:27:38.292008818Z"
    }
}

```

Success (HTTP 200 — With vault token, no expiry specified — defaults to 39 days):

```

json
{
    "api_version": "v1",
    "data": {
        "message": "success",
        "token": "pm_aee077b4-b415-48b9-a420-1417f8697344",
        "msg_id": "7WRSR6AQCQ8NQ8O7OW8QOP7298DT8YR9VAS",
        "trace_reference": "aa2fcc0f-e2e3-4768-9860-b1fe8acd2071",
        "uetr": "a6a343a2-09cf-420d-8944-8d0b3e4d261e",
        "status": "P_INITIATED",
        "order_id": "order234",
        "created_at": "2025-07-29T05:37:33.554740323Z"
    }
}

```

> Poll or await webhooks using the same GET endpoint described in Section 8a.

Webhook Events to Expect — RTP Later

- `payment.created`

- `payment.pending_authorization`
- `payment.authorized`
- `payment.completed`
- `payment.settled`
- `payment.rejected`
- `payment.failed`
- `payment.voided`
- `payment.reversed`
- `payment.refunded` (if applicable)
- `payment.refund_partial` (if applicable)

Troubleshooting — RTP Later

- **Expired request** — Regenerate with a fresh `request_id` and updated `expiry_in_days`.
- **No customer response** — Offer manual refresh options in your UI. Long-lived requests may take hours.
- **Duplicate payload error** — Retry with the exact same body and `request_id`.
- `expiry_in_minutes` **sent instead of** `expiry_in_days` — RTP Later uses `expiry_in_days`. Sending `expiry_in_minutes` is ignored or may return an error.

7c. Dynamic QR

Dynamic QR generates a single-use, EMVCo-compliant QR code that embeds a specific transaction amount. The customer scans it with their banking app and approves the payment. Use this at point-of-sale terminals, online checkouts, or kiosks.

Flow Summary

1. Retrieve the `aggregator_merchant_identifier` from the merchant creation response.
2. Call `POST /v1/aggregators/{{raast-aggregator-id}}/qrs` with `type: "DYNAMIC"` and the order amount.
3. Render the returned `data.code` string as a QR image using any QR library (e.g., `qrcode.react`, `qr-image`, `qrcodejs`).
4. The customer scans and pays in their banking app. You do not know the debtor identity until the webhook arrives.
5. Listen for payment webhooks or poll `GET /v1/aggregators/{{raast-aggregator-id}}/payments/{{payment_id}}` to confirm settlement.

Create Dynamic QR

Description: Generates a single-use QR code for a specific amount. Returns an EMVCo-compliant `data.code` string ready for rendering. Each code is associated with one aggregator merchant and expires after the configured window.

Method: `POST`

Endpoint: `/v1/aggregators/{{raast-aggregator-id}}/qrs`

Authentication: Header — `X-SFPY-AGGREGATOR-SECRET-KEY: {{secret_key}}`

Request Parameters

Type	Parameter	Data Type	Mandatory	Sample Value	Description
Header	X-SFPY-AGGREGATOR-SECRET-KEY	String	M	`{{secret_key}}`	Aggregator secret key
Body	type	String	M	`"DYNAMIC"`	Must be `DYNAMIC` for per-transaction QR codes
Body	aggregator_merchant_identifier	String	M	`"am_749fe6ce-..."`	Token from merchant creation
Body	request_id	String (UUID)	O	`"3fa85f64-..."`	Recommended for idempotency; prevents duplicate

					QR creation on retries
Body	order_id	String	O	`"order_qr_001"`	Optional order reference surfaced in webhook payloads
Body	amount	Integer	O	`15000`	Amount in paisa to embed. `15000` = PKR 150.00. Omit to let the payer enter the amount.
Body	expiry_in_minutes	Integer	O	`30`	QR validity window in minutes. Default: 180

Sample Request (cURL)

```
bash
curl --request POST \
  --url "{{baseUrl}}/v1/aggregators/{{raast-aggregator-id}}/qrs" \
  --header "X-SFPY-AGGREGATOR-SECRET-KEY: {{secret_key}}" \
  --header "Content-Type: application/json" \
  --data '{
    "type": "DYNAMIC",
    "aggregator_merchant_identifier": "am_749fe6ce-91f9-4f6a-9df2-ec6ae9f75a30",
    "request_id": "{{$guid}}",
    "order_id": "order_qr_001",
    "amount": 15000,
```

```
    "expiry_in_minutes": 30
}'

```

Response Parameters

Parameter	Data Type	Mandatory	Sample Value	Description
`api_version`	String	M	`"v1"`	API version
`data.token`	String	M	`"qr_xxxxxxxx-..."`	QR code token identifier
`data.code`	String	M	`"00020101..."`	EMVCo-compliant QR string to be rendered as a QR image
`data.type`	String	M	`"DYNAMIC"`	QR type
`data.amount`	String	M	`"15000"`	Amount embedded in the QR (in paisa)
`data.status`	String	M	`"ACTIVE"`	Current QR status
`data.expires_at`	String	M	`"2025-07-29T06:00:00Z"`	Expiry timestamp
`data.created_at`	String	M	`"2025-07-29T05:30:00Z"`	Creation timestamp

Sample Response

Success (HTTP 200):

```
json
{
    "api_version": "v1",
    "data": {
        "token": "qr_a1b2c3d4-e5f6-7890-abcd-ef1234567890",
        "code":
"00020101021226580014com.raast.www0112am_749fe6ce010203040506070809005
204599953035865405150000.005802PK5913Test Merchant6008Karachi63041AB3",
        "type": "DYNAMIC",
        "amount": "15000",
        "status": "ACTIVE",
        "order_id": "order_qr_001",
        "expires_at": "2025-07-29T06:00:00Z",
        "created_at": "2025-07-29T05:30:00Z"
    }
}
```

> **Rendering:** Feed `data.code` into any QR generation library. The string is EMVCo-compliant and Raast-banking-app-compatible.

> **Status polling:** Use `GET /v1/aggregators/{{raast-aggregator-id}}/payments/{{payment_id}}` where `payment_id` is found in the webhook payload after the QR is scanned and paid.

Webhook Events to Expect — Dynamic QR

- `payment.created` — Payment initiated when customer scans
- `payment.completed` — Customer approved and payment processed
- `payment.settled` — Funds settled to merchant
- `payment.failed` — Processing failure
- `payment.rejected` — Customer abandoned or cancelled

Troubleshooting — Dynamic QR

- **Expired QR** — Dynamic QRs expire after `expiry_in_minutes`. Generate a new QR with a fresh `request_id`.
- **No payment webhook** — Confirm your webhook endpoint is registered and accessible. Poll the payment endpoint as fallback.
- **Wrong amount** — Amount must be provided in paisa (integer). `PKR 150 = 15000` paisa.

7d. Static QR

Static QR creates a reusable QR code that identifies the aggregator merchant without embedding a specific amount. The customer scans it and enters the amount in their banking app. Use this for countertop displays, printed posters, or digital invoices where a single QR serves multiple transactions.

Flow Summary

1. Retrieve the `aggregator_merchant_identifier` from the merchant creation response.
2. Call `POST /v1/aggregators/{{raast-aggregator-id}}/qrs` with `type: "STATIC"`.
3. Store the returned `data.code` and display it as a printed or digital QR.
4. For each payment made against this QR, Safepay fires a webhook with payer details.
5. Reconcile by listing QR payments via `GET /v1/aggregators/{{raast-aggregator-id}}/qrs/{{qr_id}}/payments`.

Create Static QR

Description: Generates a reusable QR code tied to an aggregator merchant. No expiry or amount is needed. Multiple payments can be made against the same QR; each payment triggers a separate webhook event.

Method: `POST`

Endpoint: `/v1/aggregators/{{raast-aggregator-id}}/qrs`

Authentication: Header — `X-SFPY-AGGREGATOR-SECRET-KEY: {{secret_key}}`

Request Parameters

Type	Parameter	Data Type	Mandatory	Sample Value	Description
Header	X-SFPY-AGGREGATOR-SECRET-KEY	String	M	`{{secret_key}}`	Aggregator secret key

Body	type	String	M	`"STATIC"`	Must be `STATIC` for reusable QR codes
Body	aggregator_merchant_identifier	String	M	`"am_749fe6ce-..."`	Token from merchant creation
Body	request_id	String (UUID)	O	`"3fa85f64-..."`	Recommended to prevent duplicate QR creation on retries

---

Sample Request (cURL)

```
bash
curl --request POST \
  --url "{{baseUrl}}/v1/aggregators/{{raast-aggregator-id}}/qrs" \
  --header "X-SFPY-AGGREGATOR-SECRET-KEY: {{secret_key}}" \
  --header "Content-Type: application/json" \
  --data '{
    "type": "STATIC",
    "aggregator_merchant_identifier": "am_749fe6ce-91f9-4f6a-9df2-ec6ae9f75a30",
    "request_id": "{{$guid}}"
}'
```

Sample Response

Success (HTTP 200):

```
json
{
    "api_version": "v1",
    "data": {
        "token": "qr_static_x1y2z3-a4b5-6789-cdef-012345678901",
        "code": "00020101021126440014com.raast.www0112am_749fe6ce020101030101520459995
```

```
3035865802PK5913Test Merchant6008Karachi6304ABCD",
        "type": "STATIC",
        "status": "ACTIVE",
        "created_at": "2025-07-29T05:00:00Z"
    }
}

```

List QR Payments

Description: Lists all payments received against a specific Static QR code. Use for reconciliation when a Static QR is deployed at a merchant location and receives multiple incoming payments.

Method: `GET`

Endpoint: `/v1/aggregators/{{raast-aggregator-id}}/qrs/{{qr_id}}/payments`

Authentication: Header — `X-SFPY-AGGREGATOR-SECRET-KEY: {{secret_key}}`

Sample Request (cURL)

```

bash
curl --request GET \
  --url "{{baseUrl}}/v1/aggregators/{{raast-aggregator-id}}/qrs/{{qr_id}}/payments" \
  --header "X-SFPY-AGGREGATOR-SECRET-KEY: {{secret_key}}"

```

Webhook Events to Expect — Static QR

- `payment.created` — Triggered for each new payment against the QR
- `payment.completed` — Payment captured
- `payment.settled` — Funds settled

Troubleshooting — Static QR

- **Overpay / underpay** — Since payers enter the amount manually, validate the webhook `amount` field against your expected invoice value.
- **Duplicate payments** — Static QRs accept multiple payments. Implement deduplication in your reconciliation logic using `order_id` or `trace_reference`.

---

8. QR Management

This section documents QR management endpoints that are supplementary to the QR creation flows in Section 8.

List QRs

Description: Returns a paginated list of QR codes created under the aggregator. Supports filtering by merchant, status, and type. Use this endpoint to audit QR inventory or retrieve QR tokens for subsequent operations.

Method: `GET`

Endpoint: `/v1/aggregators/{{raast-aggregator-id}}/qrs`

Authentication: Header — `X-SFPY-AGGREGATOR-SECRET-KEY: {{secret_key}}`

Request Parameters

Type	Parameter	Data Type	Mandatory	Sample Value	Description
Header	X-SFPY-AGGREGATOR-SECRET-KEY	String	M	`{{secret_key}}`	Aggregator secret key
Path	raast-aggregator-id	String	M	`agg_2288490a-...`	Aggregator token
Query	limit	Integer	O	`10`	Maximum results to return
Query	offset	Integer	O	`0`	Records to skip for pagination
Query	aggregator_merchant_id	String	O	`am_749fe6ce-...`	Filter by merchant
Query	type	String	O	`"STATIC"`	Filter by QR type: `STATIC`

					or `DYNAMIC`
Query	status	String	O	`"ACTIVE"`	Filter by status

Sample Request (cURL)

```
bash
curl --request GET \
  --url "{{baseUrl}}/v1/aggregators/{{raast-aggregator-id}}/qrs?limit=10&offset=0" \
  --header "X-SFPY-AGGREGATOR-SECRET-KEY: {{secret_key}}"
```

Get QR

Description: Retrieves details of a single QR code by its token. Returns the QR code string, type, status, and associated merchant details.

Method: `GET`

Endpoint: `/v1/aggregators/{{raast-aggregator-id}}/qrs/{{qr_id}}`

Authentication: Header — `X-SFPY-AGGREGATOR-SECRET-KEY: {{secret_key}}`

Request Parameters

Type	Parameter	Data Type	Mandatory	Sample Value	Description
Header	X-SFPY-AGGREGATOR-SECRET-KEY	String	M	`{{secret_key}}`	Aggregator secret key
Path	raast-aggregator-id	String	M	`agg_2288490a-...`	Aggregator token

Path	qr_id	String	M	`qr_static_x1y2z3-...`	QR code token

Sample Request (cURL)

```
bash
curl --request GET \
  --url "{{baseUrl}}/v1/aggregators/{{raast-aggregator-id}}/qrs/{{qr_id}}" \
  --header "X-SFPY-AGGREGATOR-SECRET-KEY: {{secret_key}}"

```

---

9. Payouts

Payouts allow Simpaisa to disburse funds to beneficiaries (vendors, partners, drivers, etc.) using the same aggregator credentials as pay-ins. All payouts are asynchronous; always validate the beneficiary IBAN using Title Fetch (Section 6) before submitting a payout.

Create Payout

Description: Initiates a funds transfer to a beneficiary IBAN over the Raast network. Use a unique `request_id` per payout for idempotency. The amount must be supplied as a string in PKR (not paisa). Returns initial status `P_INITIATED`; final outcome is delivered via webhook.

Method: `POST`

Endpoint: `/v1/aggregators/{{raast-aggregator-id}}/payout`

Authentication: Header — `X-SFPY-AGGREGATOR-SECRET-KEY: {{secret_key}}`

Request Parameters

Type	Parameter	Data Type	Mandatory	Sample Value	Description
Header	X-SFPY-AGGREGATOR-	String	M	`{{secret_key}}`	Aggregator secret key

	SECRET-KEY				
Body	request_id	String (UUID)	M	`"3fa85f64-..."`	UUID for idempotency. Reuse only when retrying the same payout.
Body	amount	String	M	`"200"`	Amount in PKR as a string. `"200"` = PKR 200. Note: This is PKR, not paisa.
Body	creditor_iban	String	M	`"PK89SCBL0000000123465 1601"`	Validated beneficiary IBAN to receive the funds

---

Sample Request (cURL)

```
bash
curl --request POST \
  --url "{{baseUrl}}/v1/aggregators/{{raast-aggregator-id}}/payout" \
  --header "X-SFPY-AGGREGATOR-SECRET-KEY: {{secret_key}}" \
  --header "Content-Type: application/json" \
  --data '{
    "request_id": "{{$guid}}",
    "amount": "200",
    "creditor_iban": "PK89SCBL0000001234651601"
}'
```

Response Parameters

Parameter	Data Type	Mandatory	Sample Value	Description
`api_version`	String	M	`"v1"`	API version
`data.message`	String	M	`"success"`	Creation confirmation
`data.token`	String	M	`"po_xxxxxxxx-..."`	Payout token for status tracking
`data.status`	String	M	`"P_INITIATED"`	Initial payout status
`data.trace_reference`	String	M	`"abc123-..."`	Internal reference for reconciliation
`data.request_id`	String	M	`"3fa85f64-..."`	Echoed request ID
`data.created_at`	String	M	`"2025-07-29T05:00:00Z"`	Payout creation timestamp

Sample Response

Success (HTTP 200):

```
json
{
    "api_version": "v1",
    "data": {
        "message": "success",
        "token": "po_d1e2f3a4-b5c6-7890-abcd-ef1234567890",
        "status": "P_INITIATED",
        "trace_reference": "7f3d9a12-1234-5678-abcd-90ef12345678",
        "request_id": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
        "created_at": "2025-07-29T05:00:00Z"
    }
}
```

Payout Status Reference

Status	Description	Recommended Action
`P_INITIATED`	Safepay accepted the payout and queued it for processing	Display "Processing" in dashboards
`P_RECEIVED`	Beneficiary bank is validating the transfer	No action required; continue waiting
`P_SETTLED`	Funds have reached the beneficiary	Mark payout complete and notify recipient
`P_FAILED`	Payout could not be delivered (invalid account, compliance block, insufficient balance)	Investigate failure reason, fix data, and retry with same `request_id` if appropriate

---

Operational Best Practices

- **Pre-validate beneficiaries** — Always run Title Fetch before submitting a payout to avoid `P_FAILED` outcomes from invalid IBANs.
- **Webhook reliability** — Implement exponential backoff on your webhook receiver if your endpoint experiences downtime.
- **Dual controls** — Require two-person review for high-value payouts before calling the API.
- **Ledger reconciliation** — Store both `trace_reference` and `request_id` to match Safepay webhook events to your internal ledger entries.
- **Incident alerting** — Configure automatic alerts if a payout remains in `P_RECEIVED` beyond your SLA, or if it enters `P_FAILED`.
- **Idempotent retries** — Retry transient failures using the same `request_id` and identical payload to stay idempotent.

---

10. Refunds

Refunds allow Simpaisa to return funds to a customer for a previously completed payment. Use the payment token from the original transaction to initiate a refund.

Create Refund

Description: Initiates a full or partial refund against a completed payment. The refund is processed asynchronously; final outcome is delivered via webhook events `refund.completed` or `refund.failed`.

Method: `POST`

Endpoint: `/v1/aggregators/{{raast-aggregator-id}}/payments/{{payment_token}}/refunds`

Authentication: Header — `X-SFPY-AGGREGATOR-SECRET-KEY: {{secret_key}}`

Request Parameters

Type	Parameter	Data Type	Mandatory	Sample Value	Description
Header	X-SFPY-AGGREGATOR-SECRET-KEY	String	M	`{{secret_key}}`	Aggregator secret key
Path	raast-aggregator-id	String	M	`agg_2288490a-...`	Aggregator token
Path	payment_token	String	M	`pm_68cd947a-...`	Token of the payment to refund
Body	request_id	String (UUID)	M	`"3fa85f64-..."`	UUID for idempotency
Body	amount	Integer	O	`5000`	Amount in paisa to refund. Omit for a full refund.

Body	reason	String	O	`"Customer requested return"`	Human-readable reason for the refund

---

Sample Request (cURL)

```
bash
curl --request POST \
  --url "{{baseUrl}}/v1/aggregators/{{raast-aggregator-id}}/payments/{{payment_token}}/refunds" \
  --header "X-SFPY-AGGREGATOR-SECRET-KEY: {{secret_key}}" \
  --header "Content-Type: application/json" \
  --data '{
    "request_id": "{{$guid}}",
    "amount": 5000,
    "reason": "Customer requested return"
}'
```

Response Parameters

Parameter	Data Type	Mandatory	Sample Value	Description
`api_version`	String	M	`"v1"`	API version
`data.token`	String	M	`"rf_xxxxxxxx-..."`	Refund token
`data.status`	String	M	`"REFUND_INITIATED"`	Initial refund status
`data.amount`	String	M	`"5000"`	Refund amount in paisa
`data.request_id`	String	M	`"3fa85f64-..."`	Echoed request ID
`data.created_at`	String	M	`"2025-07-29T06:00:`	Refund creation

			`00Z"`	timestamp

Sample Response

Success (HTTP 200):

```
json
{
    "api_version": "v1",
    "data": {
        "token": "rf_a1b2c3d4-e5f6-7890-abcd-ef1234567890",
        "status": "REFUND_INITIATED",
        "amount": "5000",
        "request_id": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
        "created_at": "2025-07-29T06:00:00Z"
    }
}
```

Get Refund

Description: Retrieves the current status of a refund by its token.

Method: `GET`

Endpoint: `/v1/aggregators/{{raast-aggregator-id}}/payments/{{payment_token}}/refunds/{{refund_token}}`

Authentication: Header — `X-SFPY-AGGREGATOR-SECRET-KEY: {{secret_key}}`

Sample Request (cURL)

```
bash
curl --request GET \
  --url "{{baseUrl}}/v1/aggregators/{{raast-aggregator-id}}/payments/{{payment_token}}/refunds/{{refund_token}}" \
  --header "X-SFPY-AGGREGATOR-SECRET-KEY: {{secret_key}}"
```

Refund Status Reference

Status	Description
`REFUND_INITIATED`	Refund accepted and queued
`REFUND_COMPLETED`	Funds returned to customer

`REFUND_FAILED`	Refund could not be processed
`REFUND_CANCELED`	Refund request was cancelled

---

11. Raast Merchants

A Raast Merchant is the Raast-network-registered identity that receives payments via the SBP network. Every Aggregator Merchant must be linked to one Raast Merchant before it can participate in payment flows.

KYC Requirement for New Raast Merchants

If a Raast Merchant does not yet exist for your downstream client, Safepay requires a KYC process before issuing a `raast_merchant_id`. Submit the following documentation to Safepay compliance:

1. Business registration documents
2. Beneficial ownership details
3. Expected monthly volume and average ticket size
4. Contact information for compliance escalation

Safepay will then provide the `raast_merchant_id` which you can link to your Aggregator Merchant.

Get Raast Merchant

Description: Retrieves details of a specific Raast Merchant by token.

Method: `GET`

Endpoint: `/v1/aggregators/{{raast-aggregator-id}}/raast-merchants/{{raast-merchant-id}}`

Authentication: Header — `X-SFPY-AGGREGATOR-SECRET-KEY: {{secret_key}}`

Sample Request (cURL)

```

bash
curl --request GET \
  --url "{{baseUrl}}/v1/aggregators/{{raast-aggregator-id}}/raast-merchants/{{raast-merchant-id}}" \
  --header "X-SFPY-AGGREGATOR-SECRET-KEY: {{secret_key}}"
```

```
6
```

List Raast Merchants

Description: Returns all Raast Merchants registered under SafePay. You can use this to see if your existing merchant already is onboarded on Raast, take their Raast merchant ID which is token and then use that to create your aggregator merchant below in section 12.

Method: `GET`

Endpoint: `/v1/aggregators/{{raast-aggregator-id}}/raast-merchants`

Authentication: Header — `X-SFPY-AGGREGATOR-SECRET-KEY: {{secret_key}}`

Request Parameters

Type	Parameter	Data Type	Mandatory	Sample Value	Description
Header	X-SFPY-AGGREGATOR-SECRET-KEY	String	M	`{{secret_key}}`	Aggregator secret key
Query	limit	Integer	O	`10`	Maximum records to return
Query	offset	Integer	O	`0`	Records to skip

Sample Request (cURL)

```
1
2 bash
3 curl --request GET \
4   --url "{{baseUrl}}/v1/aggregators/{{raast-aggregator-id}}/raast-
  merchants?limit=10&offset=0" \
5   --header "X-SFPY-AGGREGATOR-SECRET-KEY: {{secret_key}}"
6
```

Response Parameters

Parameter	Data Type	Mandatory	Sample Value	Description
api_version	String	M	`"v1"`	API version
data.merchants	Array	M	`[...]`	List of merchant objects
data.merchants[].id	String	M	`"3"`	Internal merchant ID
data.merchants[].token	String	M	`"mer_ad8b77c8-..."`	Unique merchant token
data.merchants[].partner_id	String	M	`"partner_37e414c3-..."`	Associated partner ID
data.merchants[].raast_record_id	String	M	`"58357275"`	Raast record identifier
data.merchants[].status	String	M	`"Active"`	Merchant status
data.merchants[].uid_type	String	M	`"NTN"`	UID type (e.g. NTN)
data.merchants[].uid_value	String	M	`"16927516"`	UID value
data.merchants[].bank_account_id	String	M	`"acc_a5de0acc-..."`	Linked bank account ID
data.merchants[].name	String	M	`"DHA City"`	Merchant name

data.merchants[].document_type	String	M	`"NTN"`	Document type
data.merchants[].document_number	String	M	`"16927516"`	Document number
data.merchants[].address_details.country	String	M	`"PK"`	Country code
data.merchants[].address_details.city	String	M	`"Karachi"`	City
data.merchants[].address_details.state_province_region	String	M	`"Sindh"`	State / Province / Region
data.merchants[].address_details.address_line	String	M	`"DHA City Karachi..."`	Full address
data.merchants[].contact_details.mobile_number	String	M	`"+9221111590590"`	Contact mobile number
data.merchants[].contact_details.email	String	M	`"dhac@dhacitykarachi.org.pk"`	Contact email address
data.merchants[].addition	String	O	`"DHA City Karachi"`	Doing-business-as name

al_details.dba				
data.merchants[].additional_details.mcc	String	O	`"6513"`	Merchant category code
data.merchants[].additional_details.lat	String	O	`""`	Latitude of merchant location
data.merchants[].additional_details.long	String	O	`""`	Longitude of merchant location
data.merchants[].additional_details_private	Object	O	`{}`	Private additional details
data.merchants[].created_at	String	M	`"2026-02-13T15:42:35Z"`	Merchant creation timestamp
data.merchants[].updated_at	String	M	`"2026-02-13T15:43:07Z"`	Last updated timestamp
data.count	String	M	`"3"`	Total number of merchants returned

Sample Response

Success (HTTP 200):

```
json
{
    "api_version": "v1",
    "data": {
        "merchants": [
            {
```

```
                "id": "3",
                "token": "mer_ad8b77c8-11ec-4f34-bfef-1d240259dea6",
                "partner_id": "partner_37e414c3-c0cc-4541-9bea-
ab5a22964bd8",
                "raast_record_id": "58357275",
                "status": "Active",
                "uid_type": "NTN",
                "uid_value": "16927516",
                "bank_account_id": "acc_a5de0acc-8c96-4766-b452-
4a7b480e087b",
                "name": "DHA City",
                "document_type": "NTN",
                "document_number": "16927516",
                "address_details": {
                    "country": "PK",
                    "city": "Karachi",
                    "state_province_region": "Sindh",
                    "address_line": "DHA City Karachi, KM 50, Motorway
M-9, Karachi-75340, Pakistan"
                },
                "contact_details": {
                    "mobile_number": "+9221111590590",
                    "email": "dhac@dhacitykarachi.org.pk"
                },
                "additional_details": {
                    "dba": "DHA City Karachi",
                    "mcc": "6513",
                    "lat": "",
                    "long": ""
                },
                "additional_details_private": {},
                "created_at": "2026-02-13T15:42:35Z",
                "updated_at": "2026-02-13T15:43:07Z"
            }
        ],
        "count": "3"
    }
}
```


---

12. Merchants (Aggregator Merchants)

Aggregator Merchants are the downstream clients Simpaisa onboards under its aggregator account. The `data.token` returned when creating a merchant is the `aggregator_merchant_identifier` used in all payment and QR API calls.

Create Merchant

Description: Creates a new Aggregator Merchant under Simpaisa's aggregator account. The returned `data.token` is the `aggregator_merchant_identifier` required for all payment, payout, and QR operations. Each merchant must be linked to a Raast Merchant before processing live payments.

Method: `POST`

Endpoint: `/v1/aggregators/{{raast-aggregator-id}}/merchants`

Authentication: Header — `X-SFPY-AGGREGATOR-SECRET-KEY: {{secret_key}}`

Request Parameters

Type	Parameter	Data Type	Mandatory	Sample Value	Description
Header	X-SFPY-AGGREGATOR-SECRET-KEY	String	M	`{{secret_key}}`	Aggregator secret key
Body	name	String	M	`"My Merchant"`	Merchant display name
Body	raast_merchant_id	String	M	`"RM_12345"`	Raast Merchant token to link to this aggregator merchant
Body	email	String	O	`"merchant@example.com"`	Merchant contact email
Body	phone	String	O	`"03001234567"`	Merchant contact phone number

Sample Request (cURL)

```
bash
curl --request POST \
  --url "{{baseUrl}}/v1/aggregators/{{raast-aggregator-id}}/merchants" \
  --header "X-SFPY-AGGREGATOR-SECRET-KEY: {{secret_key}}" \
```

```
  --header "Content-Type: application/json" \
  --data '{
    "name": "My Merchant",
    "raast_merchant_id": "RM_12345",
    "email": "merchant@example.com",
    "phone": "03001234567"
}'

```

Response Parameters

Parameter	Data Type	Mandatory	Sample Value	Description
`api_version`	String	M	`"v1"`	API version
`data.token`	String	M	`"am_97a8f3d5-ba90-..."`	**Aggregator merchant identifier** — use this in all payment calls
`data.name`	String	M	`"My Merchant"`	Merchant name
`data.is_active`	Boolean	M	`true`	Whether the merchant is active
`data.raast_merchant_id`	String	M	`"RM_12345"`	Linked Raast Merchant ID
`data.created_at`	String	M	`"2025-07-29T05:00:00Z"`	Creation timestamp

Sample Response

Success (HTTP 200 / 201):

```

```

```
json
{
    "api_version": "v1",
    "data": {
        "token": "am_97a8f3d5-ba90-4992-8687-2a13a180f4cd",
        "name": "My Merchant",
        "is_active": true,
        "raast_merchant_id": "RM_12345",
        "email": "merchant@example.com",
        "created_at": "2025-07-29T05:00:00Z"
    }
}
```

> **Key note:** The `data.token` value (`am_...`) is used as `aggregator_merchant_identifier` in all RTP and QR payment creation calls. Store this securely per merchant record.

Get Merchant

Description: Retrieves details of a specific Aggregator Merchant by token.

Method: `GET`

Endpoint: `/v1/aggregators/{{raast-aggregator-id}}/merchants/{{aggregator-merchant-id}}`

Authentication: Header — `X-SFPY-AGGREGATOR-SECRET-KEY: {{secret_key}}`

Sample Request (cURL)

```

bash
curl --request GET \
  --url "{{baseUrl}}/v1/aggregators/{{raast-aggregator-id}}/merchants/{{aggregator-merchant-id}}" \
  --header "X-SFPY-AGGREGATOR-SECRET-KEY: {{secret_key}}"

```

List Merchants

Description: Returns all Aggregator Merchants under the aggregator. Supports pagination.

Method: `GET`

Endpoint: `/v1/aggregators/{{raast-aggregator-id}}/merchants`

Authentication: Header — `X-SFPY-AGGREGATOR-SECRET-KEY: {{secret_key}}`

Request Parameters

Type	Parameter	Data Type	Mandatory	Sample Value	Description
Header	X-SFPY-AGGREGATOR-SECRET-KEY	String	M	`{{secret_key}}`	Aggregator secret key
Query	limit	Integer	O	`10`	Maximum records to return
Query	offset	Integer	O	`0`	Records to skip
Query	is_active	Boolean	O	`true`	Filter by active status

Sample Request (cURL)

```
bash
curl --request GET \
  --url "{{baseUrl}}/v1/aggregators/{{raast-aggregator-id}}/merchants?limit=10&offset=0" \
  --header "X-SFPY-AGGREGATOR-SECRET-KEY: {{secret_key}}"

```

Update Merchant

Description: Updates metadata for an existing Aggregator Merchant.

Method: `PUT`

Endpoint: `/v1/aggregators/{{raast-aggregator-id}}/merchants/{{aggregator-merchant-id}}`

Authentication: Header — `X-SFPY-AGGREGATOR-SECRET-KEY: {{secret_key}}`

Sample Request (cURL)

```
bash
curl --request PUT \
  --url "{{baseUrl}}/v1/aggregators/{{raast-aggregator-id}}/merchants/{{aggregator-merchant-id}}" \
  --header "X-SFPY-AGGREGATOR-SECRET-KEY: {{secret_key}}" \
  --header "Content-Type: application/json" \
  --data '{
    "name": "Updated Merchant Name",
    "is_active": true
}'
```

---

13. Webhooks

Webhooks are the **primary source of truth** for all asynchronous payment outcomes. Simpaisa must maintain a publicly accessible HTTPS endpoint to receive Safepay event notifications.

13a. Create Webhook

Description: Registers a new webhook endpoint to receive event notifications. Specify the list of event types to subscribe to. The response includes a `webhook_secret` used to verify incoming payloads.

Method: `POST`

Endpoint: `/v1/aggregators/{{raast-aggregator-id}}/webhooks`

Authentication: Header — `X-SFPY-AGGREGATOR-SECRET-KEY: {{secret_key}}`

Request Parameters

Type	Parameter	Data Type	Mandatory	Sample Value	Description
Header	X-SFPY-AGGREGATOR-SECRET-KEY	String	M	`{{secret_key}}`	Aggregator secret key
Body	url	String	M	`"https://api.simpaisa`	HTTPS endpoint to receive

				`.com/webhooks/safepay"`	event payloads
Body	description	String	O	`"Raast webhook endpoint"`	Human-readable description
Body	event_types	Array	M	`["payment.created", "payment.completed"]`	List of event types to subscribe to
Body	enabled	Boolean	M	`true`	Whether this webhook is active

Sample Request (cURL)

```
bash
curl --request POST \
  --url "{{baseUrl}}/v1/aggregators/{{raast-aggregator-id}}/webhooks" \
  --header "X-SFPY-AGGREGATOR-SECRET-KEY: {{secret_key}}" \
  --header "Content-Type: application/json" \
  --data '{
    "url": "https://api.simpaisa.com/webhooks/safepay",
    "description": "Raast webhook endpoint",
    "event_types": ["payment.created", "payment.completed", "payment.settled", "refund.completed"],
    "enabled": true
}'
```

Response Parameters

Parameter	Data Type	Mandatory	Sample Value	Description

`api_version`	String	M	`"v1"`	API version
`data.token`	String	M	`"wh_xxxxxxxx-..."`	Webhook endpoint token
`data.url`	String	M	`"https://api.simpaisa.com/..."`	Registered endpoint URL
`data.event_types`	Array	M	`["payment.completed"]`	Subscribed event types
`data.enabled`	Boolean	M	`true`	Active status
`data.webhook_secret`	String	M	`"whsec_abc123..."`	Secret for HMAC signature verification — store immediately
`data.created_at`	String	M	`"2025-07-29T05:00:00Z"`	Creation timestamp

13b. List Webhooks

Description: Returns all webhook endpoints registered for the aggregator.

Method: `GET`

Endpoint: `/v1/aggregators/{{raast-aggregator-id}}/webhooks`

Authentication: Header — `X-SFPY-AGGREGATOR-SECRET-KEY: {{secret_key}}`

Sample Request (cURL)

```
bash
curl --request GET \
  --url "{{baseUrl}}/v1/aggregators/{{raast-aggregator-id}}/webhooks" \
  --header "X-SFPY-AGGREGATOR-SECRET-KEY: {{secret_key}}"

```

13c. Webhook Headers

Every event delivery from Safepay includes the following HTTP headers:

Header	Description
`X-SFPY-SIGNATURE`	HMAC-SHA256 signature of the payload for authenticity verification
`X-SFPY-TIMESTAMP`	ISO 8601 timestamp of when the event was dispatched
`X-SFPY-EVENT-ID`	Unique identifier for the event delivery
`X-SFPY-EVENT-TYPE`	Event type string (e.g., `payment.completed`)
`X-SFPY-AGGREGATOR-ID`	Your aggregator ID, confirming the intended recipient

13d. Signature Verification

Safepay signs each webhook payload using HMAC-SHA256. Always verify signatures before processing any event to prevent spoofed requests.

Verification algorithm:

1. Extract `X-SFPY-SIGNATURE` and `X-SFPY-TIMESTAMP` from the incoming request headers.
2. Construct the signing string: `timestamp + "." + raw_payload_bytes`.
3. Compute HMAC-SHA256 using your `webhook_secret` as the key.
4. Compare the computed digest to the provided `X-SFPY-SIGNATURE` using a **constant-time comparison** to prevent timing attacks.
5. Optionally enforce a timestamp tolerance (e.g., ±5 minutes) to reject replayed events.

Go reference implementation:

```
go
func Verify(secret, body []byte, providedSig, providedTS string,
tolerance time.Duration) error {
    if tolerance > 0 {
        parsed, err := time.Parse(time.RFC3339Nano, providedTS)
        if err != nil {
            return err
        }
        if delta := time.Since(parsed); delta > tolerance || delta < -
tolerance {
            return errTimestampDrift
        }
    }

    mac := hmac.New(sha256.New, secret)
    mac.Write([]byte(providedTS))
    mac.Write([]byte{'.'})
    mac.Write(body)
    expected := fmt.Sprintf(headerFormat,
hex.EncodeToString(mac.Sum(nil)))

    if !hmac.Equal([]byte(expected), []byte(providedSig)) {
        return errSignatureMismatch
    }
    return nil
}
```

> **Note:** Use the raw (unparsed) request body as the `body` argument. Parsing and re-serializing JSON can alter byte ordering and invalidate the signature.

13e. Retry Behavior

If your endpoint does not return an HTTP `200 OK` response within the timeout window, Safepay will retry the delivery using exponential backoff:

Attempt	Delay
1	1 second
2	2 seconds
3	4 seconds
4	8 seconds
5	16 seconds

After 5 failed attempts, the delivery is abandoned. Implement **idempotent processing** in your webhook handler — the same event may arrive more than once. Return `200 OK` immediately upon persisting the event to storage; process asynchronously to avoid timeouts.

13f. Full Event Catalog that you can subscribe to

Event	Category	Description
`payment.created`	Payments	A new payment request was created
`payment.pending_authorization`	Payments	Payment is awaiting customer authorization
`payment.authorized`	Payments	Payment has been authorized and funds are on hold
`payment.completed`	Payments	Payment has been captured or charged successfully
`payment.settled`	Payments	Payment funds have been settled to the merchant
`payment.refunded`	Payments	Payment was fully refunded
`payment.refund_partial`	Payments	Payment was partially refunded
`payment.rejected`	Payments	Payment was rejected before authorization
`payment.failed`	Payments	Payment processing failed
`payment.reversed`	Payments	Payment was reversed after completion
`payment.voided`	Payments	Payment authorization was voided

`settlement.created`	Settlements	Settlement request was created
`settlement.processing`	Settlements	Settlement is currently processing
`settlement.completed`	Settlements	Settlement completed successfully
`settlement.failed`	Settlements	Settlement failed during processing
`settlement.on_hold`	Settlements	Settlement temporarily placed on hold
`settlement.reversed`	Settlements	Settlement was reversed
`refund.created`	Refunds	Refund request was created
`refund.completed`	Refunds	Refund was successfully completed
`refund.failed`	Refunds	Refund failed during processing
`refund.canceled`	Refunds	The refund request was canceled

---

14. Ledger

The Ledger API provides a financial audit trail of all transactions processed under the aggregator account, including payments, payouts, refunds, and settlements. Use this for reconciliation, financial reporting, and compliance audits.

List Ledger Entries

Description: Returns a paginated list of ledger entries for the aggregator. Supports filtering by date range, merchant, transaction type, and status. Each entry represents a financial event that affected the aggregator's balance.

Method: `GET`

Endpoint: `/v1/aggregators/{{raast-aggregator-id}}/ledger`

Authentication: Header — `X-SFPY-AGGREGATOR-SECRET-KEY: {{secret_key}}`

Request Parameters

Type	Parameter	Data Type	Mandatory	Sample Value	Description
Header	X-SFPY-AGGREGATOR-SECRET-KEY	String	M	`{{secret_key}}`	Aggregator secret key
Path	raast-aggregator-id	String	M	`agg_2288490a-...`	Aggregator token
Query	limit	Integer	O	`20`	Maximum records to return. Default: 10
Query	offset	Integer	O	`0`	Records to skip for pagination
Query	aggregator_merchant_id	String	O	`am_749fe6ce-...`	Filter by specific merchant
Query	type	String	O	`"PAYMENT"`	Filter by entry type: `PAYMENT`, `PAYOUT`, `REFUND`, `SETTLEMENT`

Query	status	String	O	`"P_SETTLED"`	Filter by transaction status
Query	from_date	String	O	`"2025-07-01"`	Start date filter (ISO 8601 date)
Query	to_date	String	O	`"2025-07-31"`	End date filter (ISO 8601 date)
Query	order_id	String	O	`"order_001"`	Filter by merchant order reference
Query	trace_reference	String	O	`"abc123-..."`	Filter by internal trace reference

---

Sample Request (cURL)

```
bash
curl --request GET \
  --url "{{baseUrl}}/v1/aggregators/{{raast-aggregator-id}}/ledger?limit=20&offset=0&from_date=2025-07-01&to_date=2025-07-31" \
  --header "X-SFPY-AGGREGATOR-SECRET-KEY: {{secret_key}}"
```

Response Parameters

Parameter	Data Type	Mandatory	Sample Value	Description
`api_version`	String	M	`"v1"`	API version
`data.entries[]`	Array	M	—	Array of ledger entry

				objects
`data.entries[].id`	String	M	`"1234"`	Internal ledger entry ID
`data.entries[].token`	String	M	`"le_xxxxxxxx-..."`	Ledger entry token
`data.entries[].type`	String	M	`"PAYMENT"`	Entry type: `PAYMENT`, `PAYOUT`, `REFUND`, `SETTLEMENT`
`data.entries[].amount`	String	M	`"34000"`	Transaction amount in paisa
`data.entries[].currency`	String	M	`"PKR"`	Currency code
`data.entries[].status`	String	M	`"P_SETTLED"`	Transaction status at time of entry
`data.entries[].aggregator_merchant_id`	String	M	`"am_749fe6ce-..."`	Associated merchant token
`data.entries[].order_id`	String	O	`"order_001"`	Merchant order reference
`data.entries[].tra`	String	M	`"abc123-..."`	Internal trace reference for cross-

`ce_refere nce`				system reconciliation
`data.entr ies[].pay ment_toke n`	String	C	`"pm_68cd9 47a-..."`	Payment token (present for payment-type entries)
`data.entr ies[].cre ated_at`	String	M	`"2025-07- 29T05:00: 00Z"`	Entry creation timestamp
`data.coun t`	String	M	`"150"`	Total count of matching entries

Sample Response

Success (HTTP 200):

```
json
{
    "api_version": "v1",
    "data": {
        "entries": [
            {
                "id": "1234",
                "token": "le_a1b2c3d4-e5f6-7890-abcd-ef1234567890",
                "type": "PAYMENT",
                "amount": "34000",
                "currency": "PKR",
                "status": "P_SETTLED",
                "aggregator_merchant_id": "am_749fe6ce-91f9-4f6a-9df2-ec6ae9f75a30",
                "order_id": "test_nowExpiry",
                "trace_reference": "cb08be1d-4728-4433-85a8-01bb8ae3061b",
                "payment_token": "pm_68cd947a-03c7-40b1-88f8-bd7be978da98",
                "created_at": "2025-07-29T05:30:38Z"
            },
            {
                "id": "1235",
                "token": "le_b2c3d4e5-f6a7-8901-bcde-f12345678901",
                "type": "PAYOUT",
                "amount": "20000",
                "currency": "PKR",
                "status": "P_SETTLED",
                "aggregator_merchant_id": "am_749fe6ce-91f9-4f6a-9df2-ec6ae9f75a30",
                "order_id": null,
```

```
                "trace_reference": "7f3d9a12-1234-5678-abcd-
90ef12345678",
                "payment_token": null,
                "created_at": "2025-07-29T06:00:00Z"
            }
        ],
        "count": "150"
    }
}

```


> **Reconciliation tip:** Cross-reference `trace_reference` in ledger entries with the same field in webhook payloads and payout responses to achieve end-to-end transaction matching across Safepay, your internal ledger, and bank statements.